情報ダウンロード画面UI設計書

メタデータ

title: 情報ダウンロード画面UI設計書
category: UI設計
version: 1.0.0
last_updated: 2026-02-26
author: Development Team
tags: [provider, download, csv, ui]

関連ドキュメント

- 情報ダウンロードAPI仕様
- API一覧
- セキュリティ規約

---

1. 概要

1.1. 目的

プロバイダーが全組織の統計情報（組織・ユーザー・プロジェクト等）をCSV形式でダウンロードする画面を定義します。

1.2. 画面の特徴

- **シンプルな単一操作画面**: ページはヘッダーカードのみで構成され、「CSVダウンロード」ボタン1つで操作が完結します
- **provider 専用画面**: provider ロールのユーザーのみがアクセスできます

- **全組織データ取得**: マルチテナント分離の対象外の provider 専用 API を使用して全組織の統計データを取得します

1.3. アクセス権限

項目	内容
対象ロール	provider
認証方式	JWT（Cookieセッション）
アクセスURL	`/provider/information-download`（プロバイダーメニュー配下）

---

2. 画面構成

2.1. 画面一覧

画面名	URL	説明
情報ダウンロード画面	`/provider/information-download`	CSV一括ダウンロード

---

3. 情報ダウンロード画面

3.1. 画面概要

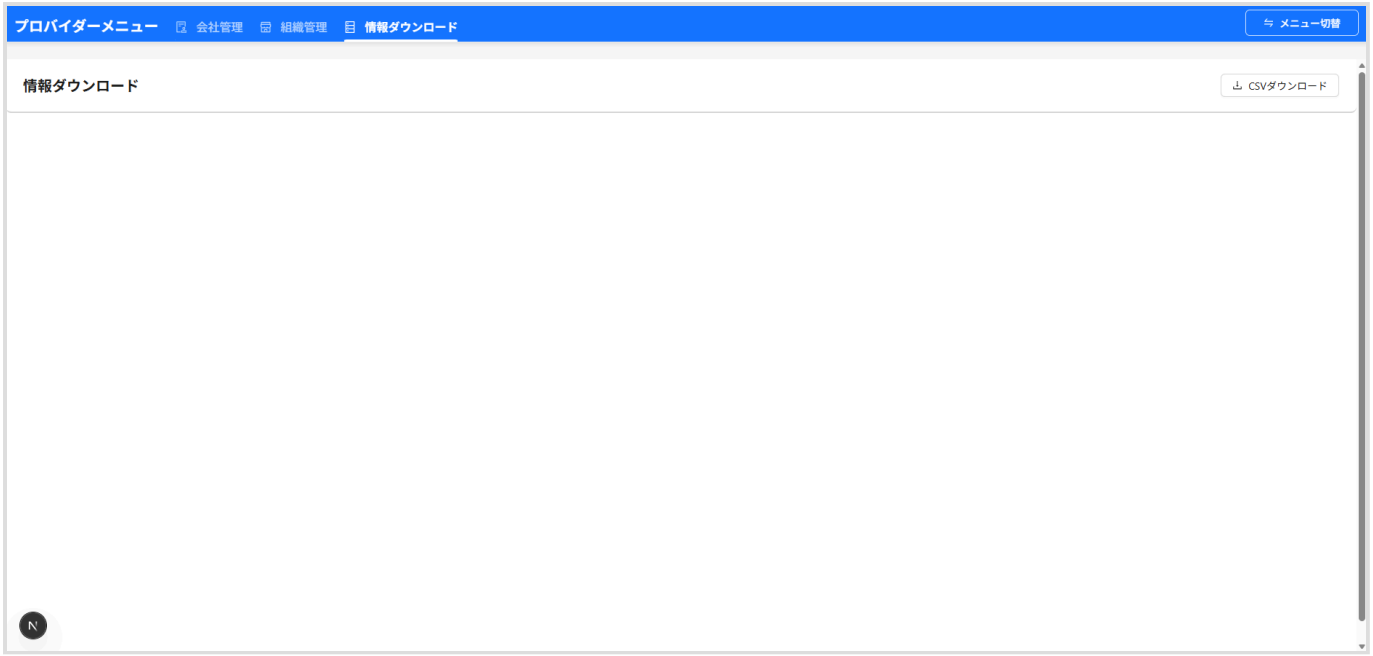
URL: `/provider/information-download`

目的: 全組織の統計情報をCSV形式でダウンロードする

前提条件:

- provider ロールでログイン済み
- セッションが有効であること

3.2. 画面全体イメージ



3.3. レイアウト

デスクトップ（lg, xl, xxl）

```
┌──────────────────────────────────────────────────────────┐
│ [サイドメニュー] │ 情報ダウンロード                      │
│                  │──────────────────────────────────────│
│ ・組織管理       │                                      │
│ ・情報ダウンロード │ ┌──────────────────────────────┐   │
│                  │ │ 情報ダウンロード  [CSVダウンロード] │ │
│                  │ └──────────────────────────────┘   │
│                  │                                      │
└──────────────────────────────────────────────────────────┘
```

ヘッダーカード詳細

```
┌──────────────────────────────────────────────────────────┐
│ 情報ダウンロード                          [↓ CSVダウンロード] │
└──────────────────────────────────────────────────────────┘
```

- **左側**: 画面タイトル「情報ダウンロード」
- **右側**: 「CSVダウンロード」ボタン（`DownloadOutlined` アイコン付き）

3.4. コンポーネント構成

```
InformationDownload (React.FC)
└── <div> (ページラッパー, padding: 0, minHeight: calc(100vh - 112px))
    └── <Card> (ヘッダーカード, variant="borderless", borderRadius: 0)
        └── <Card.body> (flexbox, space-between, items-center)
            ├── <Typography.Title level={4}> 情報ダウンロード
            └── <Button> CSVダウンロード
                    └── <DownloadOutlined />
```

3.5. コンポーネント仕様

ページラッパー

項目	値
コンポーネント	`<div>`
padding	`0`
minHeight	`calc(100vh - 112px)`

ヘッダーカード

項目	値
コンポーネント	`<Card>`
variant	`borderless`
borderRadius	`0`
boxShadow	`none`
marginBottom	`0`

Card.body スタイル:

項目	値
display	`flex`
justifyContent	`space-between`

alignItems	`center`
padding	`token.paddingMD px token.paddingLG px`
background	`token.colorBgContainer`
borderBottom	`2px solid token.colorBorder`

画面タイトル

項目	値
コンポーネント	`<Typography.Title>`
level	`4`
テキスト	「情報ダウンロード」
margin	`0`

---

4. アクションボタン仕様

4.1. CSVダウンロードボタン

コンポーネント: `<Button>`

項目	値
ラベル	「CSVダウンロード」
タイプ	`default`
アイコン	`<DownloadOutlined />`
borderRadius	`token.borderRadius`
ローディング	ダウンロード処理中は `true`
無効化	ダウンロード処理中は `true`

動作フロー:

```
[CSVダウンロードボタン] クリック
    ↓
[ローディング状態に変更]
```

```
    ↓
[API呼び出し] GET /api/v1/provider/organization/download-information-csv
    ↓ 成功
[Blob作成（text/csv）]
    ↓
[ファイル名取得]
  ├─ Content-Disposition ヘッダーからファイル名を抽出（優先）
  └─ 取得できない場合: フロントエンド側でタイムスタンプを生成
       → information_YYYYMMDDHHMMSS.csv
    ↓
[aタグ経由でダウンロード実行]
    ↓
[成功メッセージ表示] "{filename} をダウンロードしました"
    ↓
[ローディング状態を解除]
```

エラーフロー:

```
[API呼び出し]
    ↓ エラー
[エラーメッセージ表示] "CSVダウンロードに失敗しました"
    ↓
[ローディング状態を解除]
```

---

5. メッセージ仕様

5.1. 成功メッセージ

コンポーネント: `message.success`（Ant Design App.useApp）

項目	値
表示条件	CSVダウンロード成功時
メッセージ	`{filename} をダウンロードしました`
表示方式	画面上部に一時表示（自動消去）

例:

```
information_20260226174530.csv をダウンロードしました
```

5.2. エラーメッセージ

コンポーネント: `message.error`（Ant Design App.useApp）

エラー条件	メッセージ
APIエラー（認証エラー含む）	「CSVダウンロードに失敗しました」
ネットワークエラー	「CSVダウンロードに失敗しました」

---

6. APIリクエスト仕様

6.1. エンドポイント

`GET /api/v1/provider/organization/download-information-csv`

6.2. リクエスト

クエリパラメータ: なし
リクエストボディ: なし
リクエストヘッダー: Cookie（セッションCookie）

```
const response = await client.get(
  '/provider/organization/download-information-csv',
  { responseType: 'blob' }
);
```

6.3. レスポンス（成功時）

ヘッダー	値
Content-Type	`text/csv; charset=utf-8`
Content-Disposition	`attachment; filename=information_YYYYMMDDHHMMSS.csv`

6.4. CSVカラム構成

カラム名	説明
organization_id	組織ID（UUID）
organization_cd	組織コード
organization_display_name	組織表示名
corporation_name	会社名
is_active	アクティブ状態（true/false）
user_count	総ユーザー数
active_user_count	アクティブユーザー数
project_count	総プロジェクト数
created_at	組織作成日時

6.5. ファイル名形式

```
information_YYYYMMDDHHMMSS.csv
例: information_20260226174530.csv
```

ファイル名取得優先順位:

1. レスポンスヘッダーの `Content-Disposition` からファイル名を抽出
2. 取得できない場合はフロントエンドで `information_YYYYMMDDHHMMSS.csv` を生成

6.6. エラーレスポンス

ステータスコード	エラーコード	メッセージ
401 Unauthorized	`UNAUTHORIZED`	認証エラー
403 Forbidden	`FORBIDDEN`	権限不足（provider以外）
500 Internal Server Error	`SYSTEM_ERROR`	サーバー内部エラー

---

7. セキュリティ考慮事項

- `authMiddleware` によるJWT認証必須

- provider 権限のユーザーのみアクセス可能
- 全組織データを取得するためマルチテナント分離の対象外（provider専用API）
- ダウンロードされるCSVに全組織の機密統計情報が含まれるため、権限管理を厳格に行う

---

8. 実装例（TypeScript + Next.js + Ant Design）

```tsx
// frontend/src/components/Provider/InformationDownload.tsx
'use client';

import React, { useState } from 'react';
import { Card, Button, App, Typography, theme } from 'antd';
import { DownloadOutlined } from '@ant-design/icons';
import { createApiClient } from '@/utils/authUtils';

/**
 * 情報ダウンロード画面
 * バックエンドAPI経由で全組織の統計情報をCSVダウンロードする
 */
const InformationDownload: React.FC = () => {
  const { token } = theme.useToken();
  const { message: messageApi } = App.useApp();
  const [loading, setLoading] = useState(false);

  /**
   * CSVダウンロード処理
   * GET /api/v1/provider/organization/download-information-csv
   */
  const handleDownload = async () => {
    try {
      setLoading(true);
      const client = createApiClient();

      const response = await client.get(
        '/provider/organization/download-information-csv',
        { responseType: 'blob' }
      );

      const blob = new Blob([response.data], { type: 'text/csv' });

      // Content-Disposition ヘッダーからファイル名を取得
      const contentDisposition =
        response.headers['content-disposition'] ||
        response.headers['Content-Disposition'];
```

```
      let filename = 'information.csv';

      if (contentDisposition) {
        const match = contentDisposition.match(/filename[^;=\n]*=(['"]).*?\2|[^;\n]*)/);
        if (match && match[1]) {
          filename = match[1].replace(/['"]/g, '');
        }
      } else {
        // フォールバック: フロントエンド側でタイムスタンプを生成
        const now = new Date();
        const ts = [
          now.getFullYear(),
          String(now.getMonth() + 1).padStart(2, '0'),
          String(now.getDate()).padStart(2, '0'),
          String(now.getHours()).padStart(2, '0'),
          String(now.getMinutes()).padStart(2, '0'),
          String(now.getSeconds()).padStart(2, '0'),
        ].join('');
        filename = `information_${ts}.csv`;
      }

      const url = URL.createObjectURL(blob);
      const link = document.createElement('a');
      link.href = url;
      link.download = filename;
      document.body.appendChild(link);
      link.click();
      document.body.removeChild(link);
      URL.revokeObjectURL(url);

      messageApi.success(`${filename} をダウンロードしました`);
    } catch (error) {
      console.error('CSV download error:', error);
      messageApi.error('CSVダウンロードに失敗しました');
    } finally {
      setLoading(false);
    }
  };

  return (
    <div style={{ padding: 0, minHeight: 'calc(100vh - 112px)' }}>
      {/* ヘッダーカード */}
      <Card
        variant="borderless"
        style={{
          borderRadius: 0,
          marginBottom: 0,
          boxShadow: 'none',
        }}
        styles={{
```

```
          body: {
            display: 'flex',
            justifyContent: 'space-between',
            alignItems: 'center',
            padding: `${token.paddingMD}px ${token.paddingLG}px`,
            background: token.colorBgContainer,
            borderBottom: `2px solid ${token.colorBorder}`,
            marginBottom: 0,
          },
        }}
      >
        <Typography.Title level={4} style={{ margin: 0 }}>
          情報ダウンロード
        </Typography.Title>
        <Button
          type="default"
          icon={<DownloadOutlined />}
          onClick={handleDownload}
          loading={loading}
          style={{
            borderRadius: token.borderRadius,
          }}
        >
          CSVダウンロード
        </Button>
      </Card>
    </div>
  );
};

export default InformationDownload;
```

---

9. テスト要件

9.1. 単体テスト（Unit Test）

テスト対象

- `handleDownload` 関数
- ファイル名取得ロジック（`Content-Disposition` ヘッダーあり/なし）
- ローディング状態管理

テストケース例

```
describe('InformationDownload', () => {
  test('CSVダウンロードボタンクリックでAPIが呼び出される', async () => {
    // APIモックを設定
    // ボタンをクリック
    // API呼び出しが行われることを確認
  });

  test('Content-Dispositionヘッダーからファイル名を取得できる', async () => {
    // Content-Disposition: attachment; filename=information_20260226174530.csv
    // ファイル名が正しく抽出されることを確認
  });

  test('Content-Dispositionヘッダーがない場合にフォールバックファイル名を生成する', async () => {
    // ヘッダーなしのレスポンスをモック
    // information_YYYYMMDDHHmmss.csv 形式のファイル名が生成されることを確認
  });

  test('APIエラー時にエラーメッセージが表示される', async () => {
    // APIエラーをモック
    // "CSVダウンロードに失敗しました" が表示されることを確認
  });

  test('ダウンロード処理中はボタンがローディング状態になる', async () => {
    // API呼び出し中のローディング状態を確認
  });
});
```

9.2. 統合テスト（Integration Test）

テストシナリオ

1. **正常ダウンロードフロー**
    - provider でログイン
    - 情報ダウンロード画面を表示
    - 「CSVダウンロード」ボタンをクリック
    - CSVファイルがダウンロードされることを確認
    - 成功メッセージが表示されることを確認

2. **認証エラー**
    - 未ログイン状態でAPIを呼び出す
    - 401 エラーが返却される
    - エラーメッセージが表示されることを確認
3. **権限エラー**
    - provider 以外のロールでAPIを呼び出す
    - 403 エラーが返却される
    - エラーメッセージが表示されることを確認

9.3. 手動テスト

No	テスト内容	期待結果
1	provider でログインし情報ダウンロード画面を表示する	画面が正常に表示される
2	「CSVダウンロード」ボタンをクリックする	ダウンロードダイアログが表示され、CSVファイルがダウンロードされる
3	ダウンロード後に成功メッセージを確認する	`{filename} をダウンロードしました` が表示される
4	ダウンロードされたCSVのカラムを確認する	organization_id, organization_cd, organization_display_name, corporation_name, is_active, user_count, active_user_count, project_count, created_at の各カラムが含まれる
5	ダウンロード中にボタンがローディング状態になることを確認する	ボタンがスピナー表示になり、再クリックできない
6	provider 以外のロールでページにアクセスする	アクセスが拒否される（リダイレクトまたはエラー表示）

---

10. 変更履歴

バージョン	日付	変更内容	担当者
1.0.0	2026-02-26	初版作成	Development Team